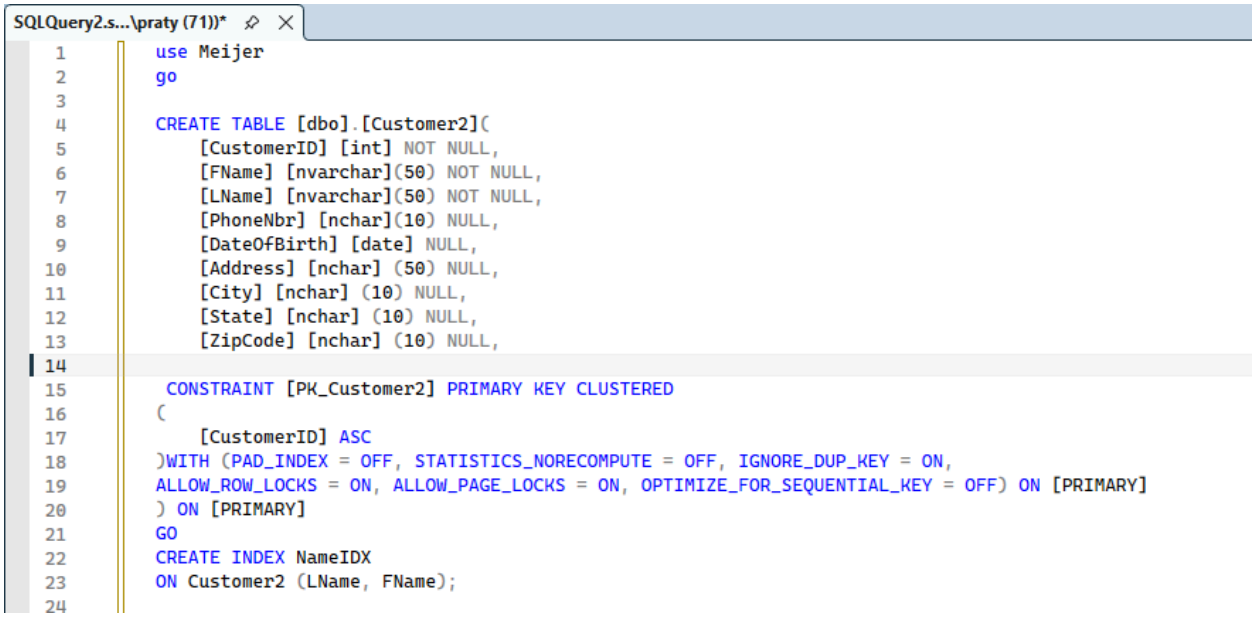
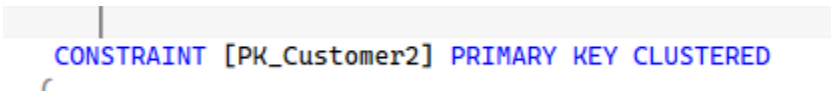
Overview

This SQL 3 assignment helped me learn some practical skills by implementing, testing, and documenting a set of database tasks. It shows table design with appropriate data types and identity/ constrains, index creation, data population and cleanup (both truncate and drop), view creation and usage, control-flow routines (IF, ELSEIF, ELSE). This assignment really helped me learn about duplicates, producing a summary of duplicate key values, and extracting the actual duplicate rows. However, this assignment was also one of the hardest one I have done so far. I face a significant problem when working with Error Logs and Error Checking. It is also worth noting that you have to let 'Identity Specification' to 'Yes' in-order to capture the errors. One worthly skill I developed was about #17, and something to work again is about #14.

1) SQL to create a Customer2 table using appropriate data types that has the following columns: CustomerID, First Name, Last Name, Date of Birth, Address, City, State, Zip Code. CustomerID must be defined as an identity field that starts with 100.



2) SQL to define the primary key in the CustomerID column. (SQL must not be generated)



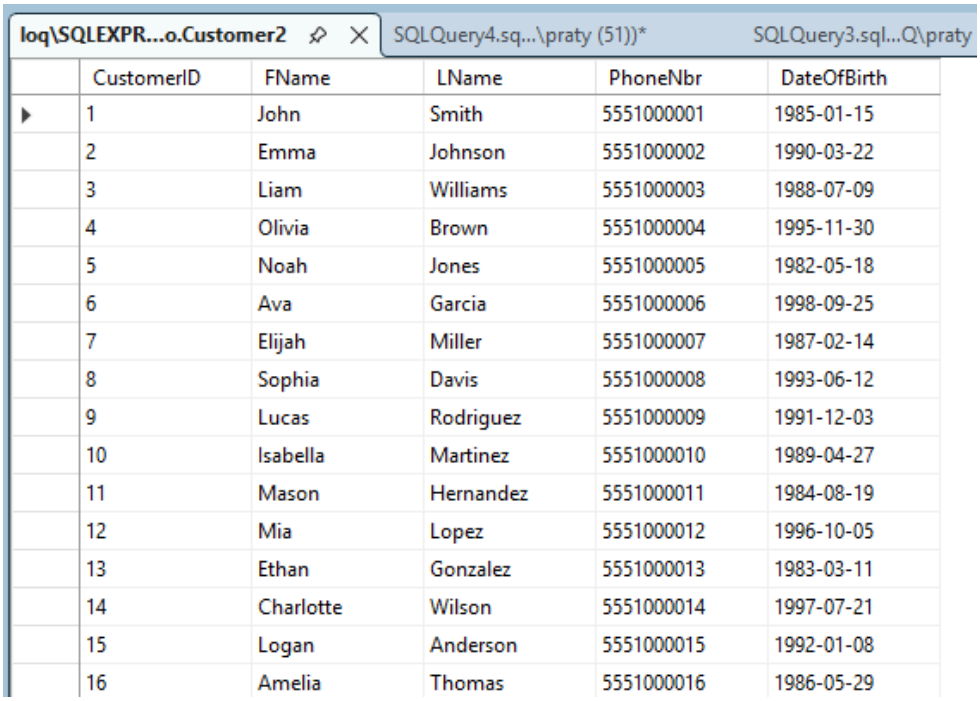
3) SQL to define an index using the primary key and a second index using Last Name and First Name. (SQL must not be generated)

```
 CONSTRAINT [PK_Customer2] PRIMARY KEY CLUSTERED
(
    [CustomerID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = ON,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]
GO
CREATE INDEX NameIDX
ON Customer2 (LName, FName);
```

4) SQL to insert fifty rows of unique data into the Customer2 table

```
INSERT INTO [dbo].[Customer2] (CustomerID, FName, LName, PhoneNbr, DateOfBirth)
VALUES
(1, 'John', 'Smith', '5551000001', '1985-01-15'),
(2, 'Emma', 'Johnson', '5551000002', '1990-03-22'),
(3, 'Liam', 'Williams', '5551000003', '1988-07-09'),
(4, 'Olivia', 'Brown', '5551000004', '1995-11-30'),
(5, 'Noah', 'Jones', '5551000005', '1982-05-18'),
(6, 'Ava', 'Garcia', '5551000006', '1998-09-25'),
(7, 'Elijah', 'Miller', '5551000007', '1987-02-14'),
```

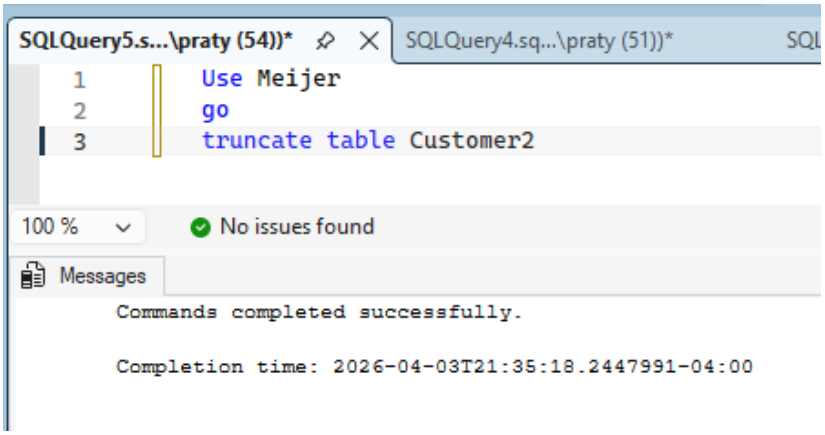
.

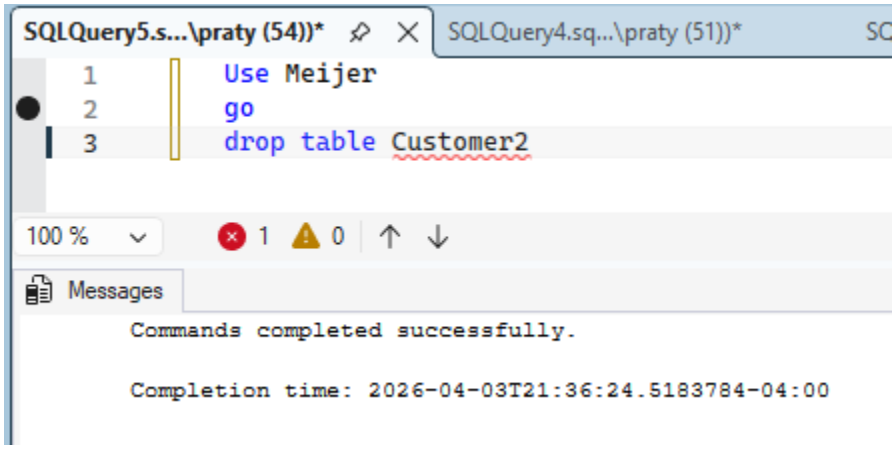


loq\SQLEXPR...o.Customer2 | SQLQuery4.sq...\praty (51))* | SQLQuery3.sql...Q\praty

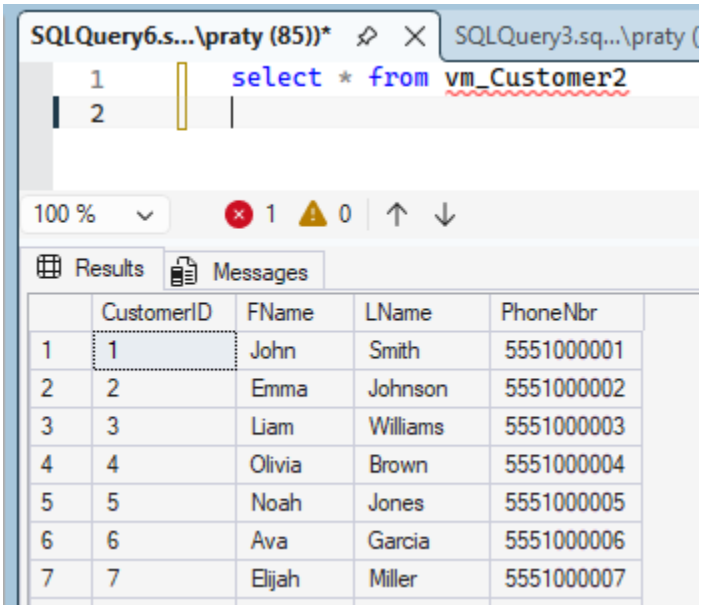
CustomerID	FName	LName	PhoneNbr	DateOfBirth
1	John	Smith	5551000001	1985-01-15
2	Emma	Johnson	5551000002	1990-03-22
3	Liam	Williams	5551000003	1988-07-09
4	Olivia	Brown	5551000004	1995-11-30
5	Noah	Jones	5551000005	1982-05-18
6	Ava	Garcia	5551000006	1998-09-25
7	Elijah	Miller	5551000007	1987-02-14
8	Sophia	Davis	5551000008	1993-06-12
9	Lucas	Rodriguez	5551000009	1991-12-03
10	Isabella	Martinez	5551000010	1989-04-27
11	Mason	Hernandez	5551000011	1984-08-19
12	Mia	Lopez	5551000012	1996-10-05
13	Ethan	Gonzalez	5551000013	1983-03-11
14	Charlotte	Wilson	5551000014	1997-07-21
15	Logan	Anderson	5551000015	1992-01-08
16	Amelia	Thomas	5551000016	1986-05-29

5) SQL to truncate and drop the Customer2 table.



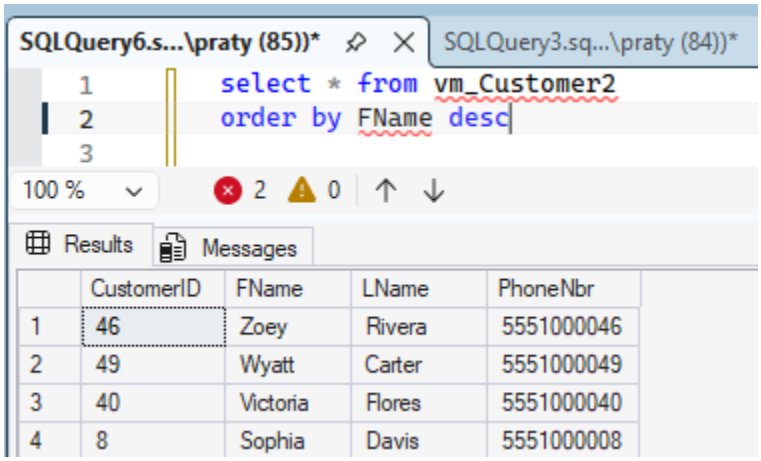


6) SQL to create a view of the Customer2 table using all of the columns in the table except Date of Birth.



	CustomerID	FName	LName	PhoneNbr
1	1	John	Smith	5551000001
2	2	Emma	Johnson	5551000002
3	3	Liam	Williams	5551000003
4	4	Olivia	Brown	5551000004
5	5	Noah	Jones	5551000005
6	6	Ava	Garcia	5551000006
7	7	Elijah	Miller	5551000007

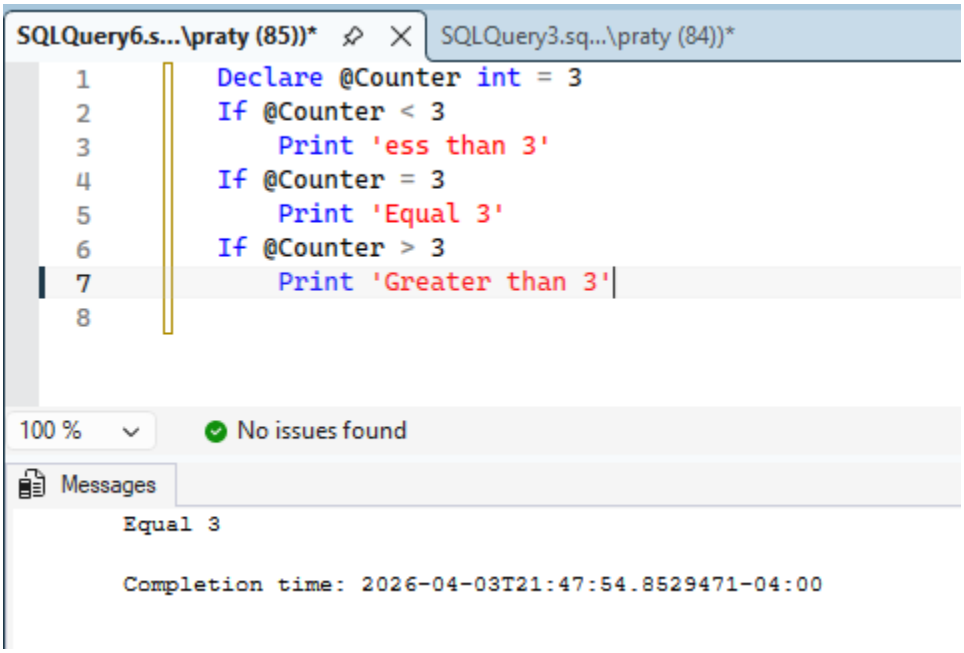
7) SQL that uses the view to display all Customers in the Customer2 table sorted by First Name in descending order.



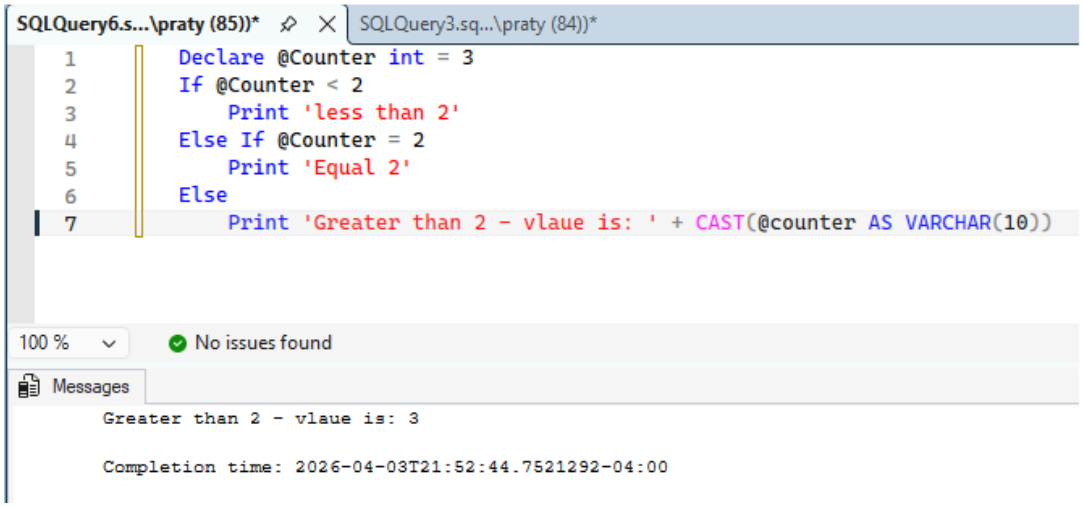
	CustomerID	FName	LName	PhoneNbr
1	46	Zoey	Rivera	5551000046
2	49	Wyatt	Carter	5551000049
3	40	Victoria	Flores	5551000040
4	8	Sophia	Davis	5551000008

Now use the Meijer database ->

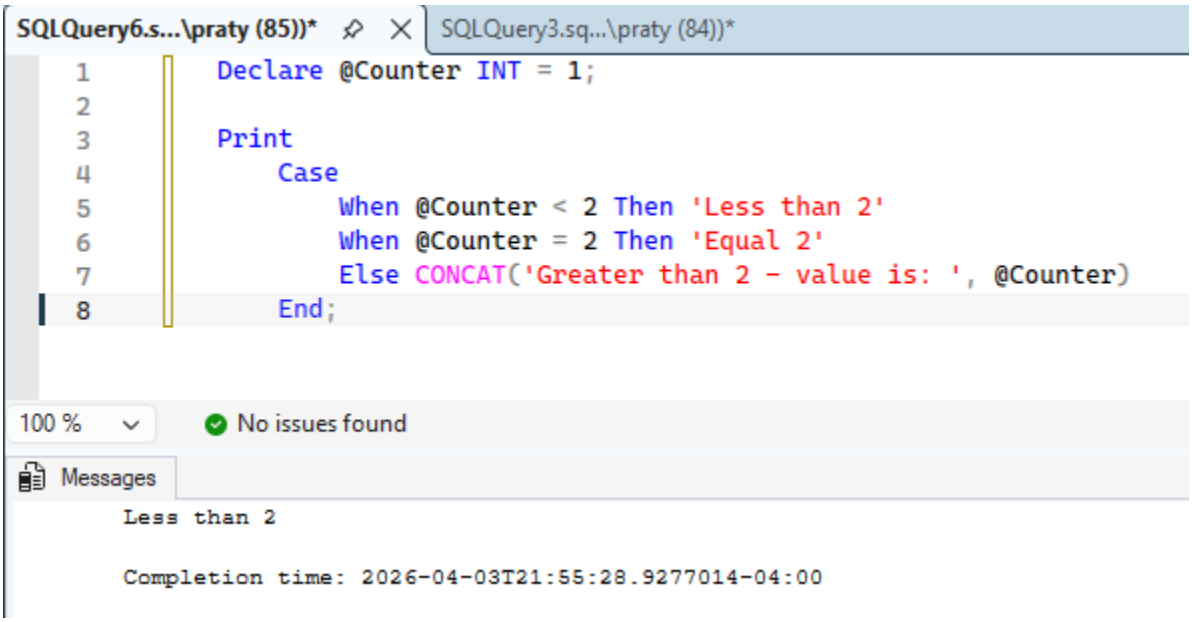
8) One sql routine with at least 3 If statements and confirmation that they work.



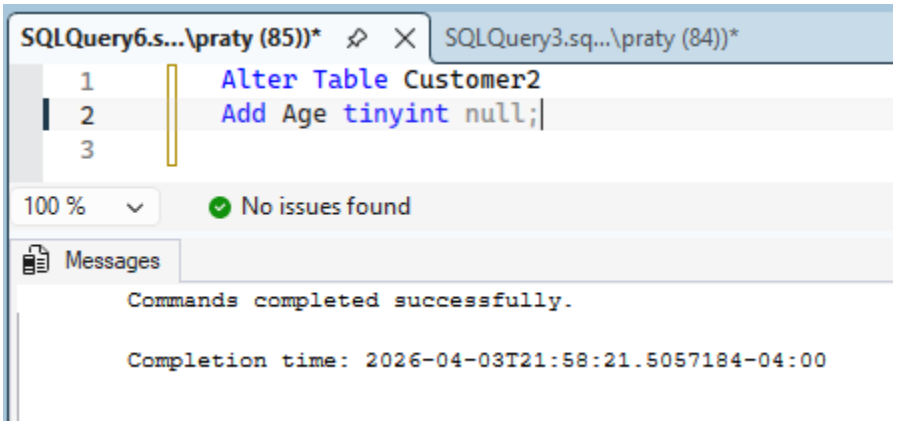
9) One sql routine with If, Else-If and confirmation that they work.



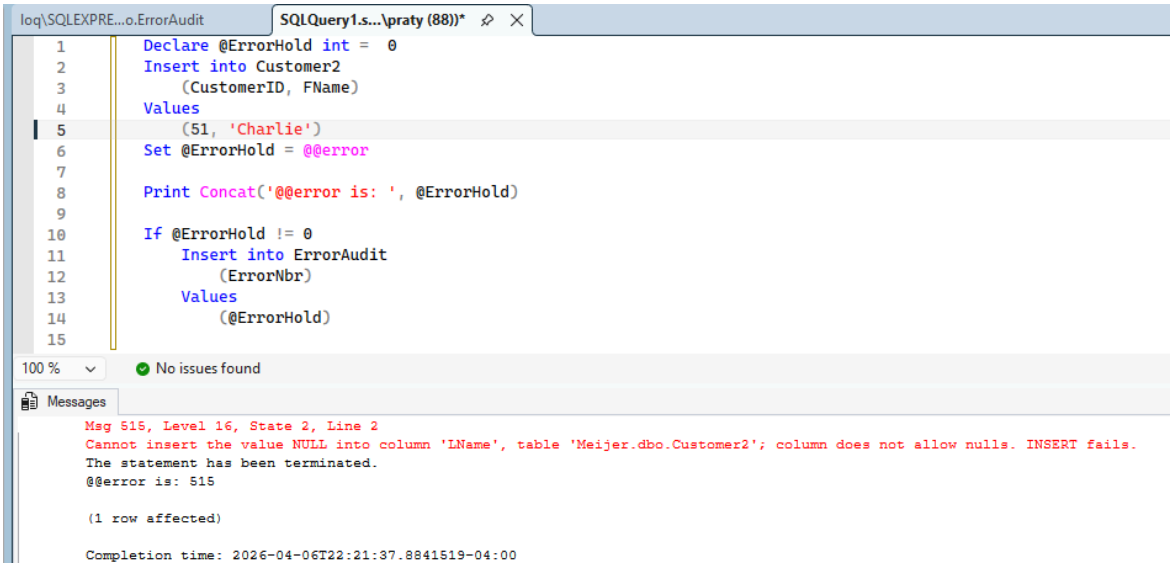
10) One Case statement with at least 3 When options and confirmation that they work.

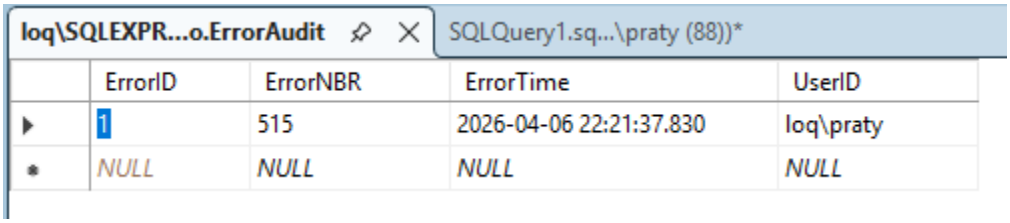


11) Alter table sql to add an integer column to an existing table that allows nulls.



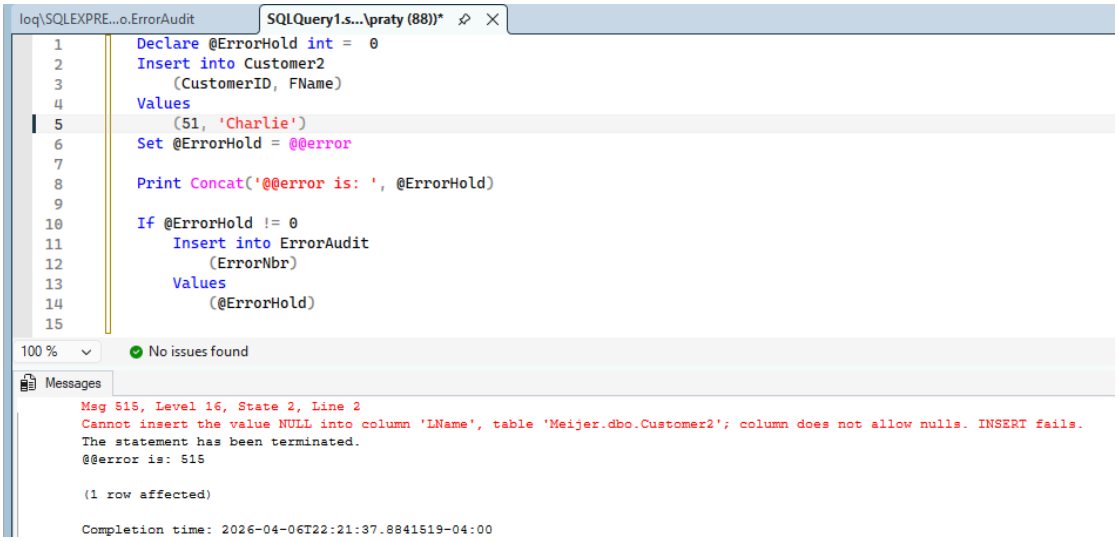
12) SQL update routine with error checking and error logging. Confirmation statement that generates logged error.





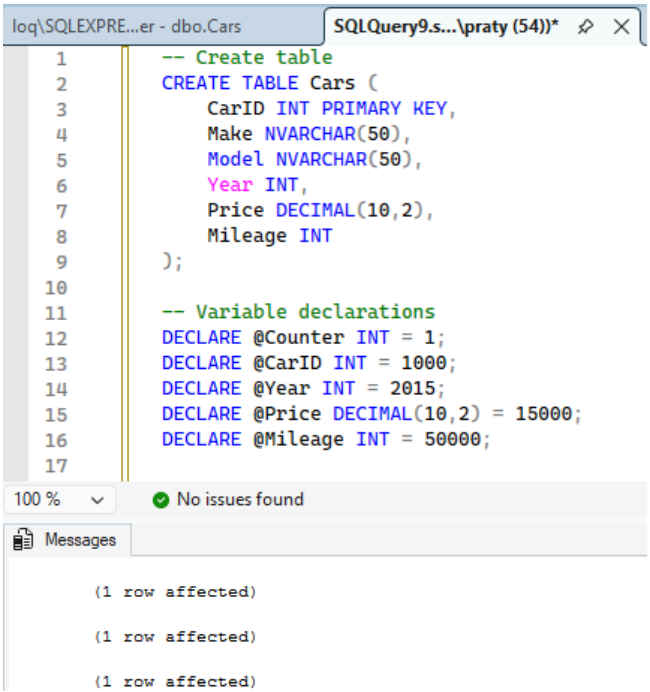
	ErrorID	ErrorNBR	ErrorTime	UserID
▶	1	515	2026-04-06 22:21:37.830	loq\praty
*	NULL	NULL	NULL	NULL

13) SQL statement that defines and uses variables.



14) Store procedure that accepts a variable and has error checking and logging as part of the execute stored procedure statement.

15) SQL insert statement that inserts ten rows of data into a new table named Cars that includes at least 5 columns and the data varies for each record by using variables that are incremented and/or changed within the routine for each record.



loq\SQLEXPRE...r - dbo.Cars | SQLQuery9.sq...\praty (54))* | loq\SQLEXPRE...o.ErrorAudit | lo

CarID	Make	Model	Year	Price	Mileage
1000	Make1	Model1	2015	15000.00	50000
1001	Make2	Model2	2016	16200.00	53500
1002	Make3	Model3	2017	17400.00	57000
1003	Make4	Model4	2018	18600.00	60500
1004	Make5	Model5	2019	19800.00	64000
1005	Make6	Model6	2020	21000.00	67500
1006	Make7	Model7	2021	22200.00	71000
1007	Make8	Model8	2022	23400.00	74500
1008	Make9	Model9	2023	24600.00	78000
1009	Make10	Model10	2024	25800.00	81500
NULL	*NULL*	*NULL*	*NULL*	*NULL*	*NULL*

16) Create a table named HikingLocations that has the appropriate structure to remove duplicates when loading the table. Demonstrate that it works. Include sql that defines the structure to remove duplicates in your submission. (this is the same as any other table create except you set the ignore_dup_key option to on).

loq\SQLEXPRE...kingLocation | SQLQuery12....\praty (51))*

```
1   Alter Table HikingLocation
2   Drop Constraint PK_HikingLocation;
3   GO
4   Create Unique Index HikingLocationIDX2
5   On HikingLocation(HikingLocationID)
6   With (IGNORE_DUP_KEY = ON);
7
8   Insert into HikingLocation
9       (HikingLocationID, Name)
10  Values
11      (1, 'Yosemite National Park'),
12      (2, 'Grand Canyon National Park'),
13      (2, 'Grand Canyon National Park'),
14      (3, 'Rocky Mountain National Park'),
15      (4, 'Glacier National Park'),
16      (5, 'Zion National Park')
```

loq\SQLEXPR...kingLocation | SQLQuery12.s...Q\praty (51))*

	HikingLocatio...	Name
▶	1	Yosemite National Park
	2	Grand Canyon National Park
	3	Rocky Mountain National Park
	4	Glacier National Park
	5	Zion National Park
*	NULL	NULL

17) Define a table named HikingLocations2 that has 10 records and 3 of the records are duplicates. Create an SQL routine that identifies the duplicate records by key value. Create a second SQL routine that displays only the duplicate records.

hint:

SELECT a, b, COUNT(*) occurrences FROM t1 GROUP BY a, b HAVING COUNT(*) > 1;

loq\SQLEXPRE...ingLocation2 | SQLQuery19....\praty (87))* | SQLQuery15.s...Q\praty (74))*

```
6   INSERT INTO HikingLocation2(HikingLocationID, Name) VALUES
7   (1, 'Yosemite National Park'),
8   (2, 'Grand Canyon National Park'),
9   (2, 'Grand Canyon National Park'),
10  (2, 'Grand Canyon National Park'),
11  (3, 'Rocky Mountain National Park'),
12  (4, 'Glacier National Park'),
13  (5, 'Zion National Park'),
14  (6, 'Great Smoky Mountains National Park'),
15  (7, 'Arches National Park'),
16  (8, 'Acadia National Park');
```

100 % | No issues found

Messages

```
(10 rows affected)

Completion time: 2026-04-03T23:28:25.5518279-04:00
```

loq\SQLEXPRE...ingLocation2 | SQLQuery19.s...Q\praty (87))* | **SQLQuery15....\praty (74))*** | SQLQ

```
1  SELECT HikingLocationID, Name, Count(*) Occurances
2  FROM HikingLocation2 GROUP BY HikingLocationID, Name HAVING COUNT(*) > 1;
3
```

100 % | 5 | 0

Results | Messages

	HikingLocationID	Name	Occurances
1	2	Grand Canyon National Park	3

loq\SQLEXPR...gLocations2 | SQLQuery19.s...Q\praty (87))* | SQLQuery1

	HikingLocationID	Name
▶	1	Yosemite National Park
	2	Grand Canyon National Park
	3	Grand Canyon National Park
	4	Grand Canyon National Park
	5	Rocky Mountain National Park
	6	Glacier National Park
	7	Zion National Park
	8	Great Smoky Mountains National Park
	9	Arches National Park
	10	Acadia National Park
*	*NULL*	*NULL*

loq\SQLEXPR...ngLocations2 | SQLQuery19.s...Q\praty (87))*

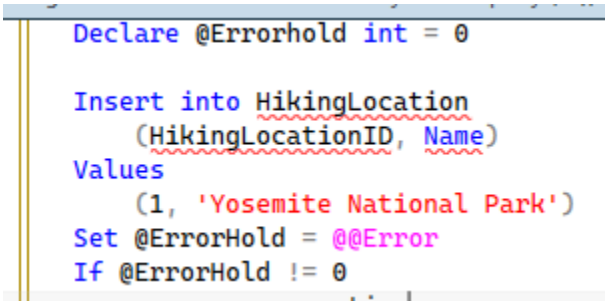
```
1  SELECT Name, Count(*) Occurances
2  FROM HikingLocations2
3  GROUP BY Name
4  --HAVING COUNT(*) > 1;
5
```

100 % | 3 | 0

Results | Messages

	Name	Occurances
1	Acadia National Park	1
2	Arches National Park	1
3	Glacier National Park	1
4	Grand Canyon National Park	3
5	Great Smoky Mountains National Park	1
6	Rocky Mountain National Park	1
7	Yosemite National Park	1
8	Zion National Park	1

18) Create an insert routine for HikingLocations2 that uses variables and performs error checking and error logging.



```
Declare @Errorhold int = 0

Insert into HikingLocation
    (HikingLocationID, Name)
Values
    (1, 'Yosemite National Park')
Set @ErrorHold = @@Error
If @ErrorHold != 0
```